

Multirate DSP:

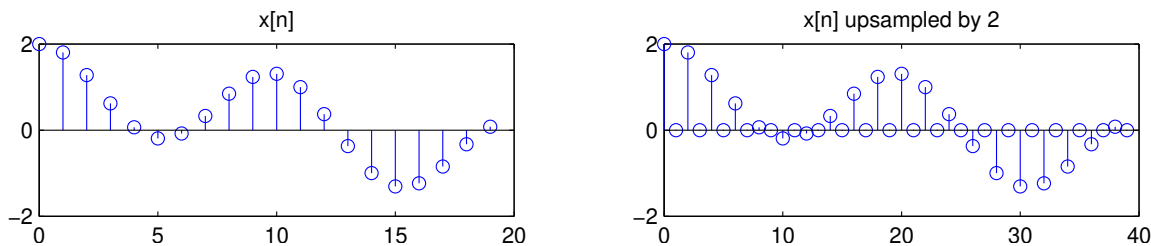
- Previously, we learned about several multirate operations:
 - downsampling
 - upsampling
 - decimation
 - interpolation
 - resampling
- We will now revisit and expand upon those operations via
 - z -domain derivations
 - filter design for interpolation/decimation
 - the Noble identities
 - polyphase interpolation, decimation, and resampling
 - allpass interpretation of ideal polyphase filters
 - multistage interpolation

Upsampling:

- Recall the upsampling operation

$$x[n] \longrightarrow \boxed{\uparrow U} \longrightarrow y[m]$$

- To upsample $x[n]$ by $U \in \mathbb{Z}^+$, we simply insert $U - 1$ zeros between each sample in $x[n]$.



- In the time domain, upsampling can be written as

$$y[m] = \begin{cases} x[\frac{m}{U}] & \text{if } \frac{m}{U} \in \mathbb{Z} \\ 0 & \text{else} \end{cases}$$

- In the DTFT domain, we previously derived that

$$Y(e^{j\omega}) = X(e^{j\omega U}) = X((e^{j\omega})^U)$$

- We can convert this to the z -domain via $e^{j\omega} \rightarrow z$:

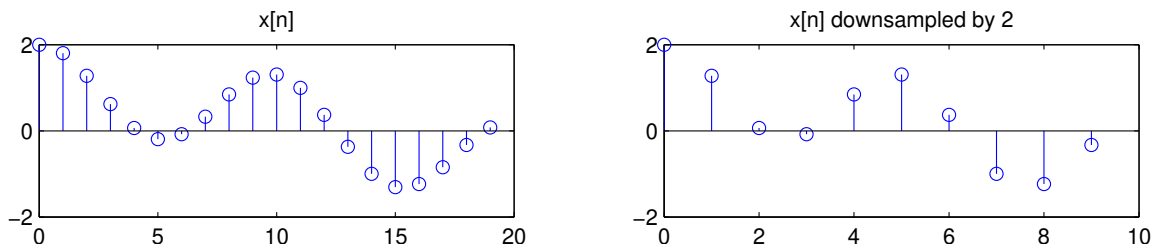
$$Y(z) = X(z^U)$$

Downsampling:

- Recall the downsampling operation

$$x[n] \longrightarrow \boxed{\downarrow D} \longrightarrow y[m]$$

- To downsample $x[n]$ by $D \in \mathbb{Z}^+$, we keep every D^{th} sample of $x[n]$ and discard the others.



- In the time domain, downsampling can be written as

$$y[m] = x[Dm]$$

- In the DTFT domain, we previously derived that

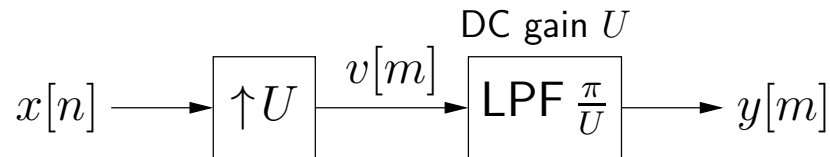
$$Y(e^{j\omega}) = \frac{1}{D} \sum_{k=0}^{D-1} X(e^{j\frac{\omega-2\pi k}{D}}) = \frac{1}{D} \sum_{k=0}^{D-1} X((e^{j\omega})^{\frac{1}{D}} e^{-j\frac{2\pi k}{D}})$$

- We can convert this to the z -domain via $e^{j\omega} \rightarrow z$:

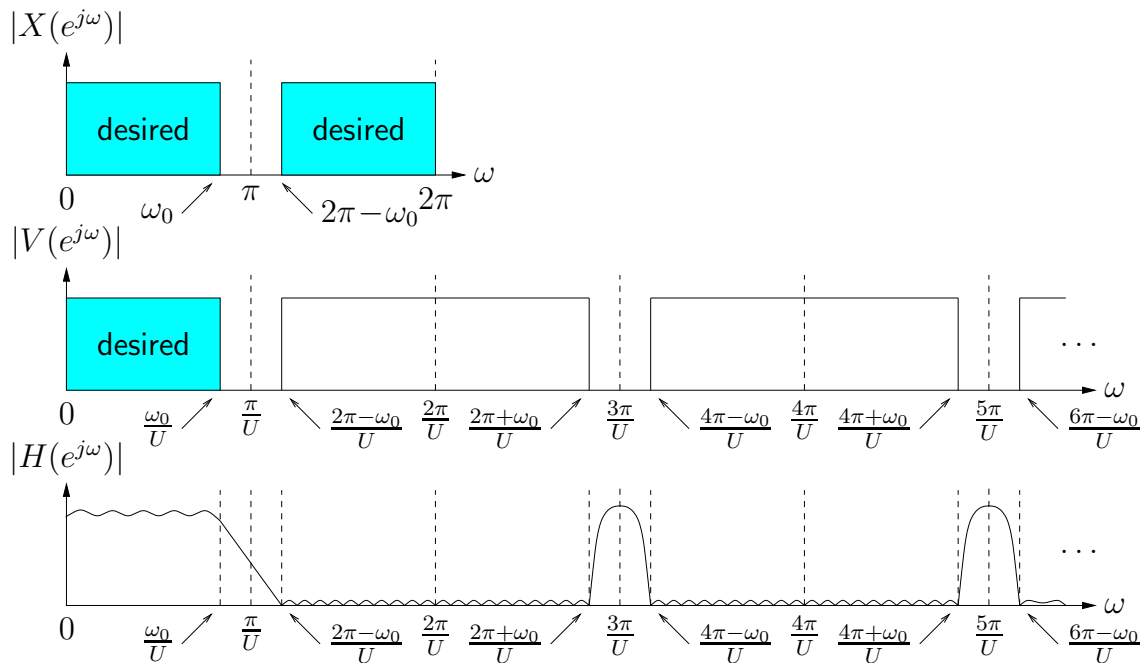
$$Y(z) = \frac{1}{D} \sum_{k=0}^{D-1} X(z^{\frac{1}{D}} e^{-j\frac{2\pi k}{D}})$$

Interpolation Filter Design:

- Recall interpolation, which increases the effective sampling rate by a factor $U \in \mathbb{Z}^+$ using



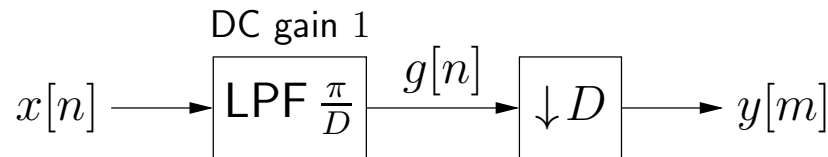
- The LPF removes unwanted spectral copies in $V(e^{j\omega})$. Let's take a closer look at the design of this LPF.
- Assuming real $x[n]$ bandlimited to ω_0 radians/sample, we can treat the dead zones in $V(e^{j\omega})$ as transition bands:



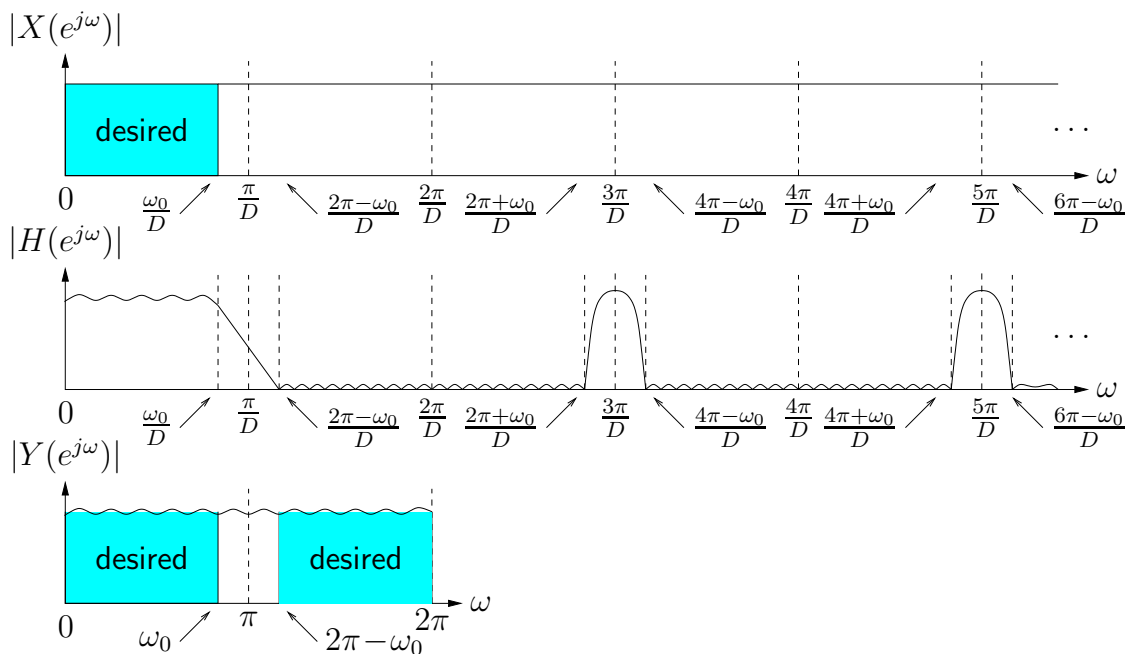
- By including these transition bands, we can design a filter with fewer coefficients and/or better ripple performance.

Decimation Filter Design:

- Recall decimation, which decreases the effective sampling rate by factor $D \in \mathbb{Z}^+$ using



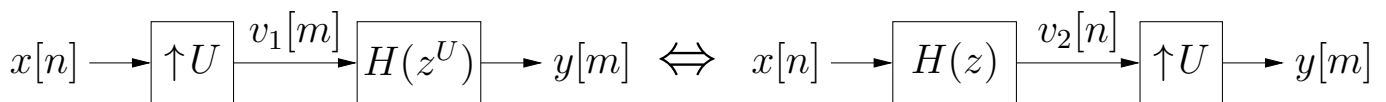
- The LPF is used to prevent aliasing when downsampling.
- Assuming we want to preserve frequencies $|\omega| \leq \frac{\omega_0}{D}$ in $X(e^{j\omega})$, we can place transition bands at frequencies that won't alias into frequencies $|\omega| \leq \omega_0$ in $Y(e^{j\omega})$:



- Note that the passband error in $Y(e^{j\omega})$ will approximately equal $\frac{1}{D}(\delta_p + (D-1)\delta_s)$, where $\frac{1}{D}$ is due to the D -downsampling, and $D-1$ is the number of stopbands in $G(e^{j\omega})$ that alias into $|\omega| \leq \omega_0$ in $Y(e^{j\omega})$.

The Noble Identities:

- The Noble identities tell us when we can swap the order of filtering and upsampling or downsampling.
- We will use them to derive efficient “polyphase” approaches to interpolation and decimation.
- The *Noble identity for upsampling* is the following:



Note that the impulse response of $H(z^U)$ is the U -upsampled version of $\{h[n]\}$.

- Proof: In the left diagram above, we have

$$Y(z) = H(z^U)V_1(z) = H(z^U)X(z^U)$$

while in the right diagram, we have

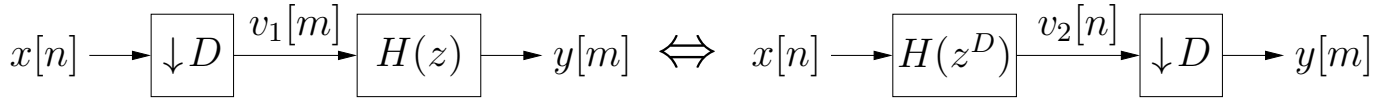
$$Y(z) = V_2(z^U) = H(z^U)X(z^U) \quad \text{since} \quad V_2(z) = H(z)X(z)$$

Thus, the left and right diagrams are equivalent.

- In words, this identity says “filtering-then-upsampling is equivalent to upsampling-then-filtering with an upsampled filter.”

The Noble Identities (cont.):

- The *Noble identity for downsampling* is the following:



Similar to before, the impulse response of $H(z^D)$ is the D -upsampled version of $\{h[n]\}$.

- Proof: In the left diagram above, we have

$$Y(z) = H(z)V_1(z) = H(z)\frac{1}{D}\sum_{k=0}^{D-1}X(e^{-j\frac{2\pi}{D}k}z^{\frac{1}{D}})$$

while in the right diagram, we have

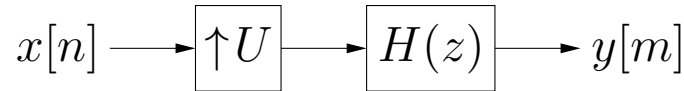
$$\begin{aligned} Y(z) &= \frac{1}{D}\sum_{k=0}^{D-1}V_2(e^{-j\frac{2\pi}{D}k}z^{\frac{1}{D}}) \text{ where } V_2(z) = X(z)H(z^D) \\ &= \frac{1}{D}\sum_{k=0}^{D-1}X(e^{-j\frac{2\pi}{D}k}z^{\frac{1}{D}})H(e^{-j2\pi k}z) \\ &= H(z)\frac{1}{D}\sum_{k=0}^{D-1}X(e^{-j\frac{2\pi}{D}k}z^{\frac{1}{D}}) \end{aligned}$$

Thus, the left and right diagrams are equivalent.

- In words, this identity says “downsampling-then-filtering is equivalent to filtering-then-downsampling with an upsampled filter.”

Polyphase Interpolation:

- The standard interpolation procedure



is computationally inefficient because the filter operates on a sequence of mostly zeros.

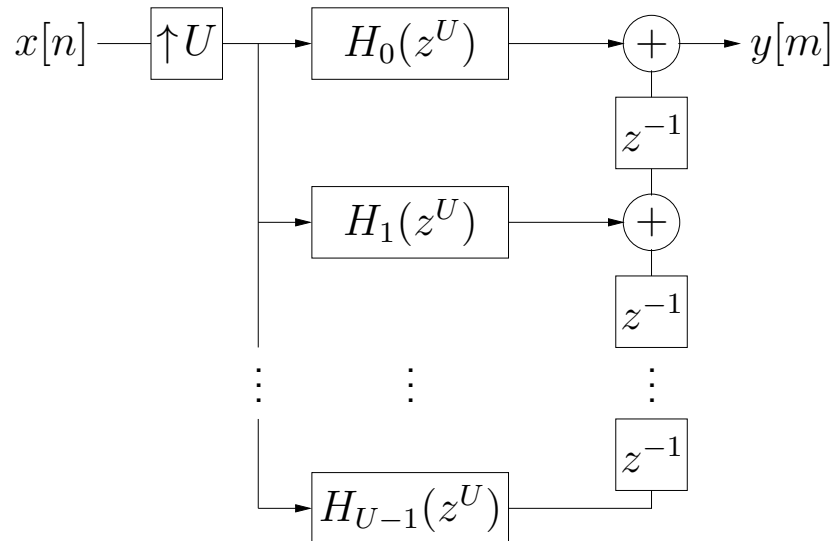
- Using the Noble identities, we can design an efficient interpolator.
- To begin, we decompose the filter $H(z)$ into its polyphase components $\{H_p(z)\}_{p=0}^{U-1}$:

$$\begin{aligned}
 H(z) &= \sum_{n \in \mathbb{Z}} h[n] z^{-n} && (U = 3) \quad \begin{array}{cccc} n = & 0 & 1 & 2 \\ & 3 & 4 & 5 & m \\ & 6 & 7 & 8 & \downarrow \\ & 9 & 10 & 11 \\ & p & \rightarrow \end{array} \\
 &= \sum_{m \in \mathbb{Z}} \sum_{p=0}^{U-1} h[mU + p] z^{-(mU+p)} \text{ via } m \triangleq \lfloor \frac{n}{U} \rfloor, p \triangleq \langle n \rangle_U \\
 &= \sum_{p=0}^{U-1} \left(\sum_{m \in \mathbb{Z}} h_p[m] z^{-mU} \right) z^{-p} \text{ via } h_p[m] \triangleq h[mU + p] \\
 &= \sum_{p=0}^{U-1} H_p(z^U) z^{-p}
 \end{aligned}$$

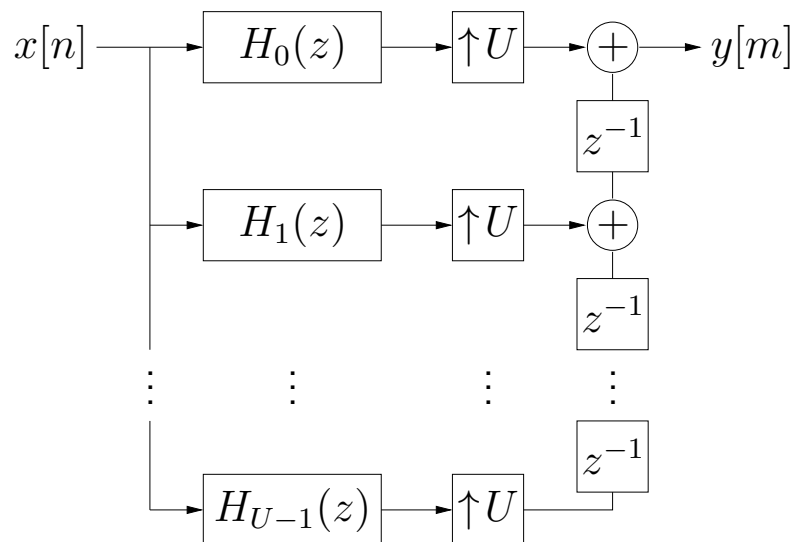
- The filter $\{h_p[m]\}_{m \in \mathbb{Z}}$ contains every U th sample of $\{h[n]\}_{n \in \mathbb{Z}}$ at offset p .

Polyphase Interpolation (cont.):

- With $H(z) = \sum_{p=0}^{U-1} H_p(z^U) z^{-p}$, interpolation becomes



- Moving $\uparrow U$ into the branches and applying the Noble identity for upsampling, we get

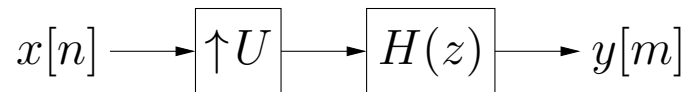


where now the filters don't operate on upsampled signals!

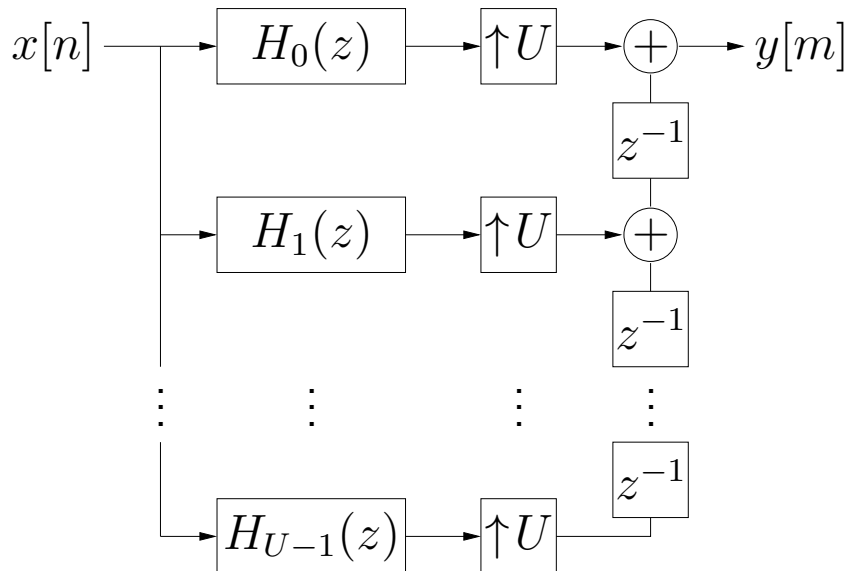
- The “ladder” of upsamplers and delays on the right performs parallel-to-serial conversion.

Computational Savings of Polyphase Interp:

- Suppose that $H(z)$ is a length- L FIR filter.
- Standard interpolation consumes L multiplies per *output*:



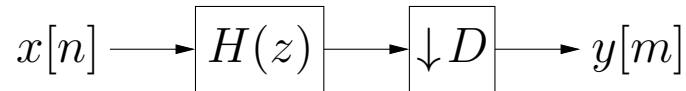
- Polyphase interpolation consumes L multiplies per *input*:



- Since there are U times as many outputs as inputs, polyphase interpolation uses a factor-of- U fewer multiplies.

Polyphase Decimation:

- The standard decimation procedure

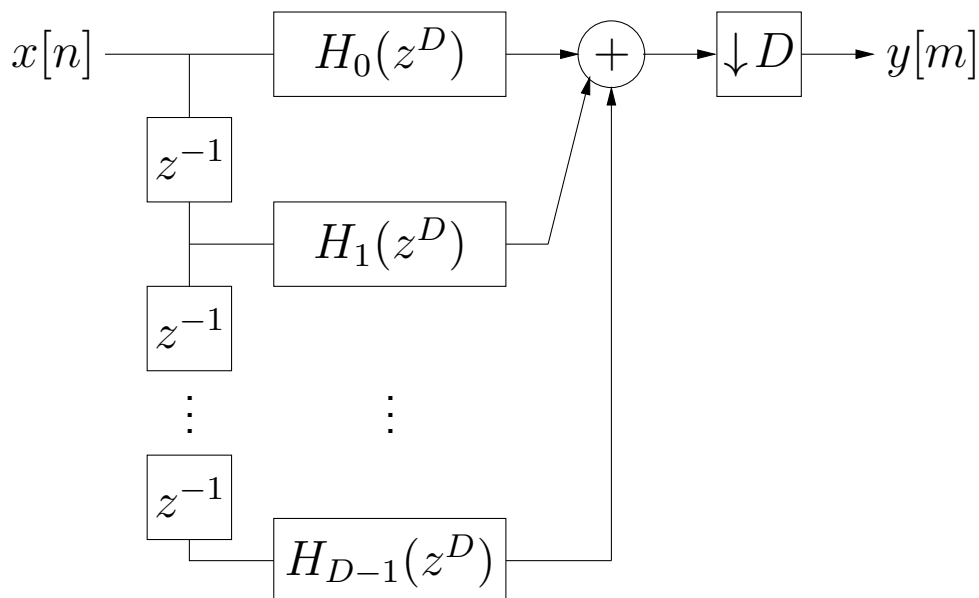


is computationally inefficient because it discards most of the filter outputs.

- Using the Noble identities, we can design an efficient decimator.
- As before, we can use the polyphase decomposition

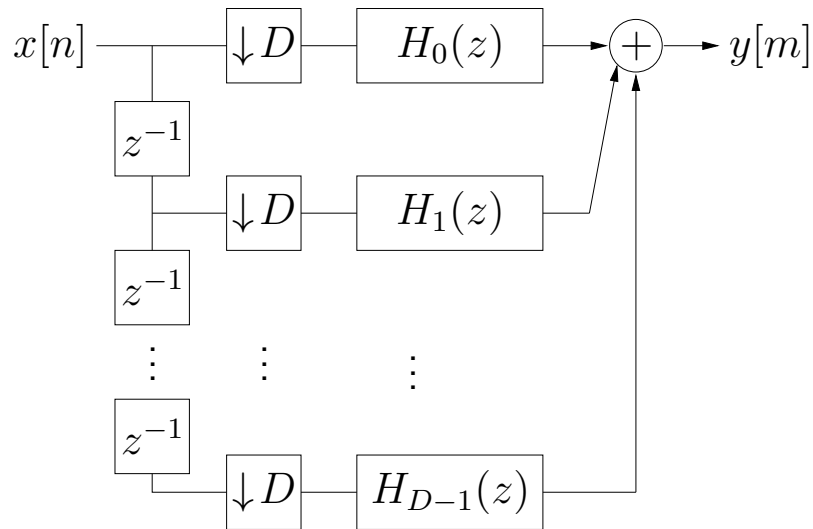
$$h_p[m] \triangleq h[mD + p] \quad \Rightarrow \quad H(z) = \sum_{p=0}^{D-1} H_p(z^D) z^{-p}$$

to redraw the diagram as



Polyphase Decimation (cont.):

- Moving $\downarrow D$ into the branches and applying the Noble identity for downsampling, we get



where no filter outputs are discarded!

- The “ladder” of delays and downsamplers on the left performs serial-to-parallel conversion.

Computational Savings:

- Polyphase decimation consumes a factor-of- D fewer multiplies than standard decimation since...
 - Standard consumes L multiplies per input
 - Polyphase consumes L multiplies per output
 - There are D times as many inputs as outputs.

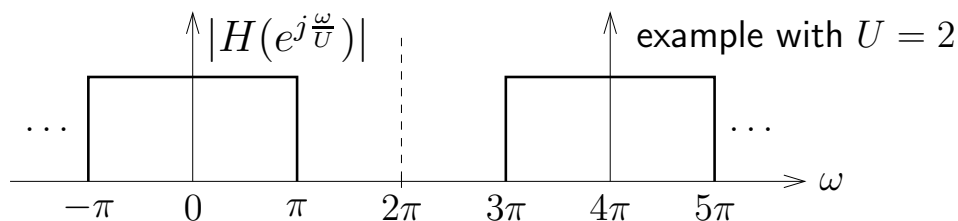
Polyphase Filters as Allpass Delays:

- Consider the *ideal* linear-phase interpolation lowpass filter:

$$H(e^{j\omega}) = \begin{cases} Ue^{-j\omega\ell} & \omega \in (-\frac{\pi}{U}, \frac{\pi}{U}] \\ 0 & \omega \in (-\pi, -\frac{\pi}{U}] \cup (\frac{\pi}{U}, \pi] \end{cases}$$

- For each p , the polyphase filter $h_p[m] = h[mU+p]$ is a U -downsampled version of p -advanced $h[n]$. Thus

$$\begin{aligned} H_p(e^{j\omega}) &= \frac{1}{U} \sum_{k=0}^{U-1} V(e^{j\frac{\omega-2\pi k}{U}}) \quad \text{where} \quad V(e^{j\omega}) = e^{j\omega p} H(e^{j\omega}) \\ &= \frac{1}{U} \sum_{k=0}^{U-1} e^{j(\frac{\omega-2\pi k}{U})p} H(e^{j\frac{\omega-2\pi k}{U}}) \\ &= \frac{1}{U} e^{j\frac{\omega}{U}p} H(e^{j\frac{\omega}{U}}) \quad \text{for} \quad |\omega| \leq \pi \\ &= e^{-j\frac{\ell-p}{U}\omega} \quad \text{for} \quad |\omega| \leq \pi. \end{aligned}$$

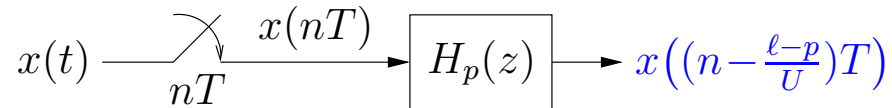


- Since $|H_p(e^{j\omega})| = 1 \quad \forall \omega$, the polyphase filters are *allpass*.
- Since $\phi(\omega) = -\frac{\ell-p}{U}\omega$ for $H_p(e^{j\omega})$, its group delay is

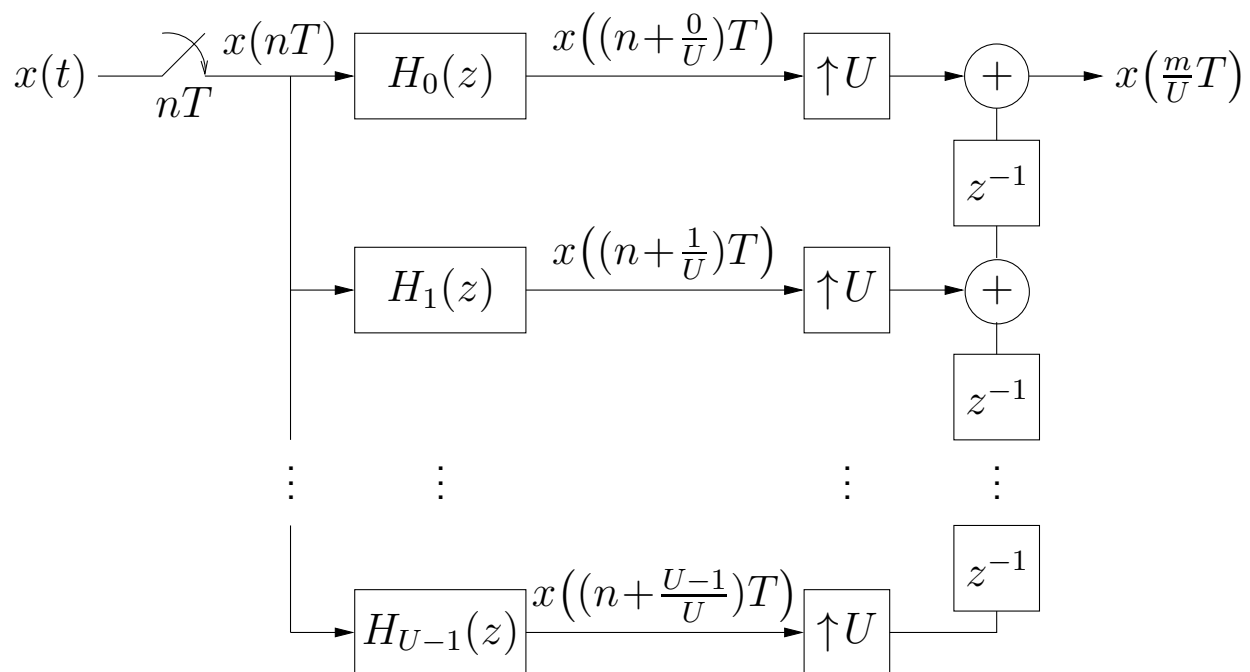
$$g(\omega) = -\frac{d\phi(\omega)}{d\omega} = \frac{\ell-p}{U},$$

Polyphase Filters as Allpass Delays (cont.):

- So the ideal polyphase filter delays by $\frac{\ell-p}{U}$ samples!



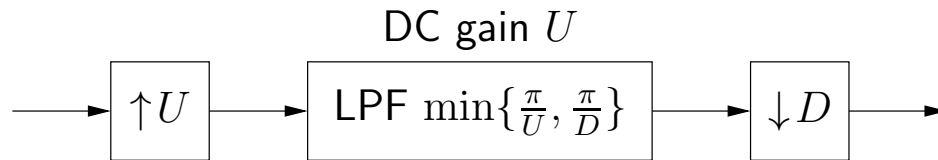
- Thus, the polyphase interpolator with ideal polyphase filters implements (using $\ell = 0$ for illustration)



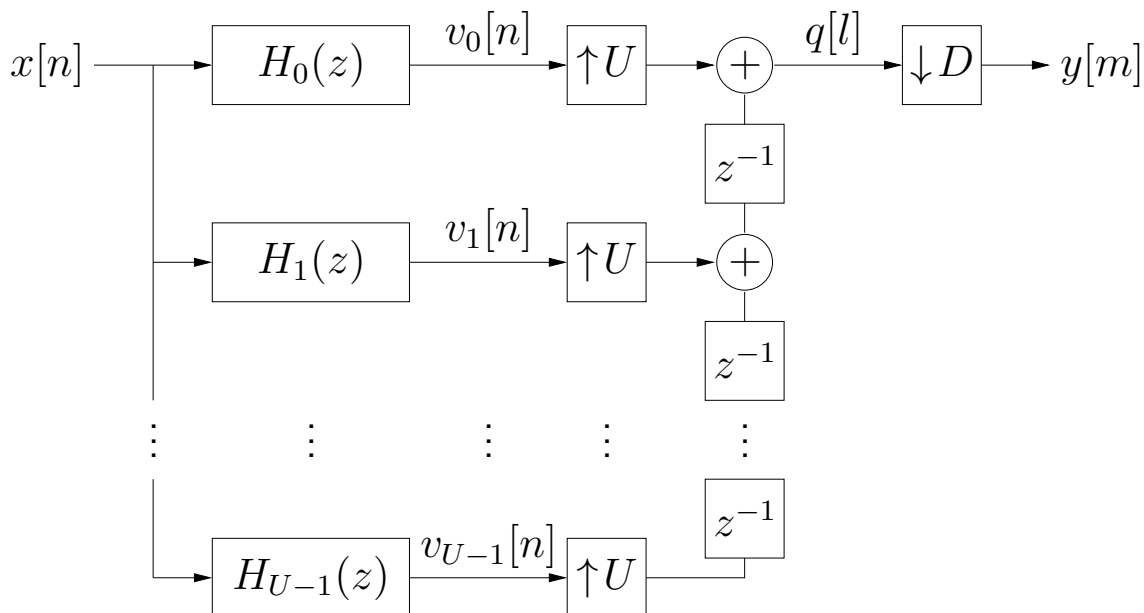
which can be recognized as ideal interpolation.

Polyphase Resampling:

- Recall that we can resample by rational rate U/D using



- Suppose that we implemented the upsampler/LPF pair using polyphase:



This is still inefficient: $\downarrow D$ throws away computations.

- Idea:
 - The ladder of upsamplers/delays performs parallel-to-serial conversion.
 - Compute only the branch that is needed at time m .

Polyphase Resampling (cont.):

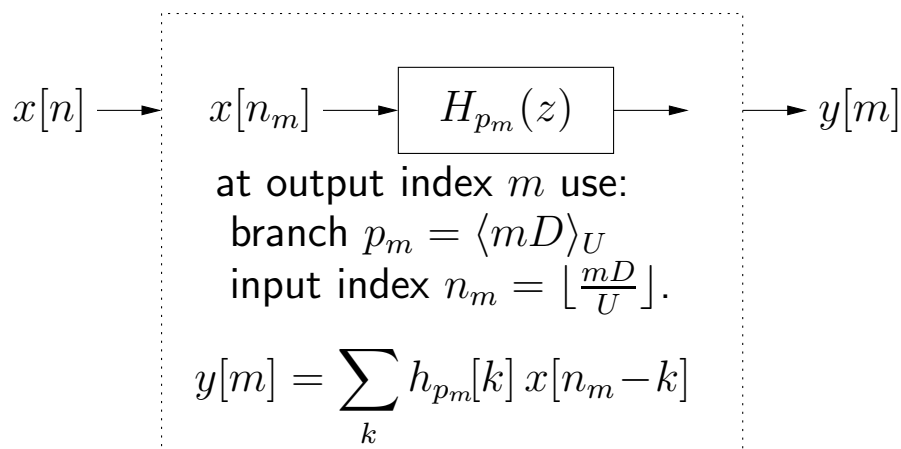
- Given the “parallel-to-serial” interpretation, the input to the downsampler, $q[l]$, takes the form

$$\begin{aligned} &\dots, v_0[0], v_1[0], v_2[0], \dots, v_{U-1}[0], \\ &\quad v_0[1], v_1[1], v_2[1], \dots, v_{U-1}[1], \\ &\quad v_0[2], v_1[2], v_2[2], \dots, v_{U-1}[2], \dots \end{aligned}$$

leading to the observation that

$$\begin{aligned} q[l] &= v_{\langle l \rangle_U} \left[\left\lfloor \frac{l}{U} \right\rfloor \right] \\ y[m] &= q[mD] \\ &= v_{\langle mD \rangle_U} \left[\left\lfloor \frac{mD}{U} \right\rfloor \right] \\ &= \sum_k h_{\langle mD \rangle_U}[k] x \left[\left\lfloor \frac{mD}{U} \right\rfloor - k \right] \end{aligned}$$

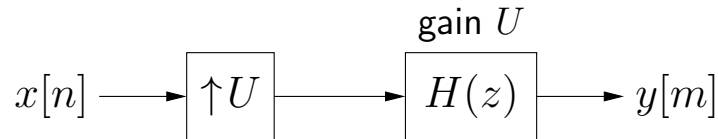
- We can summarize this procedure as follows:



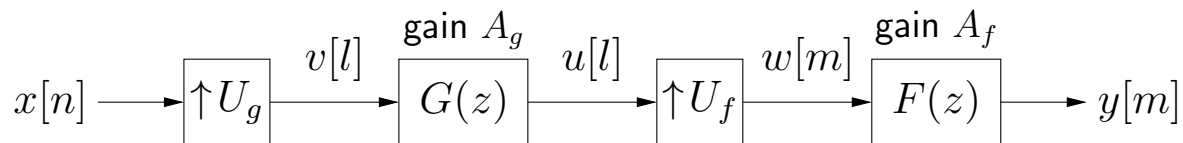
- Computational Savings: standard resampling consumes $UD\times$ as many mults as polyphase.

Multi-stage Interpolation:

- Recall the standard interpolation procedure:

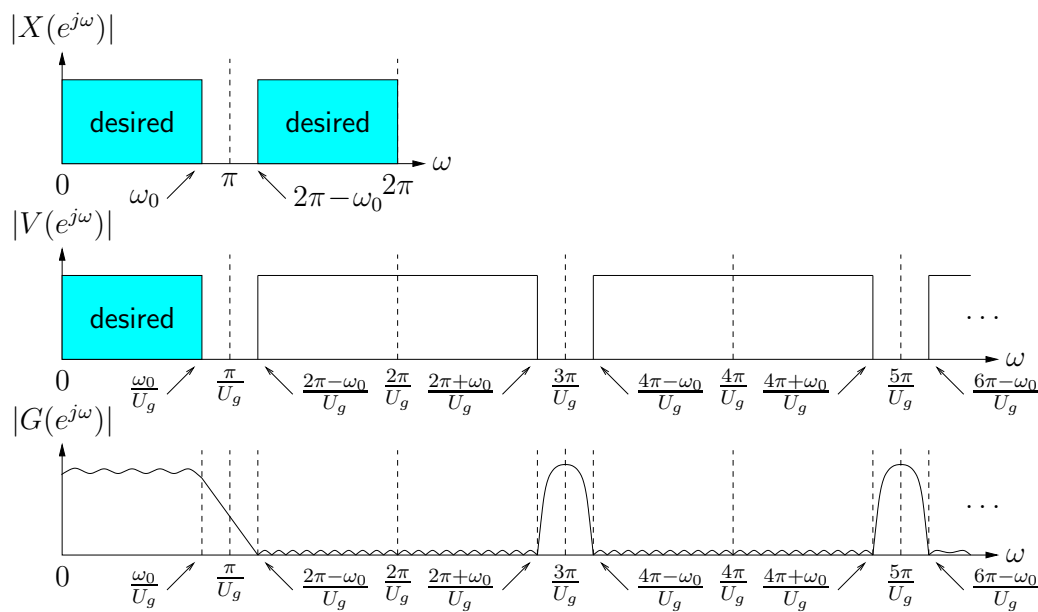


- It can be much more efficient to do this in two stages:



where $U_g U_f = U$ and $A_g A_f = U$. Let's see why...

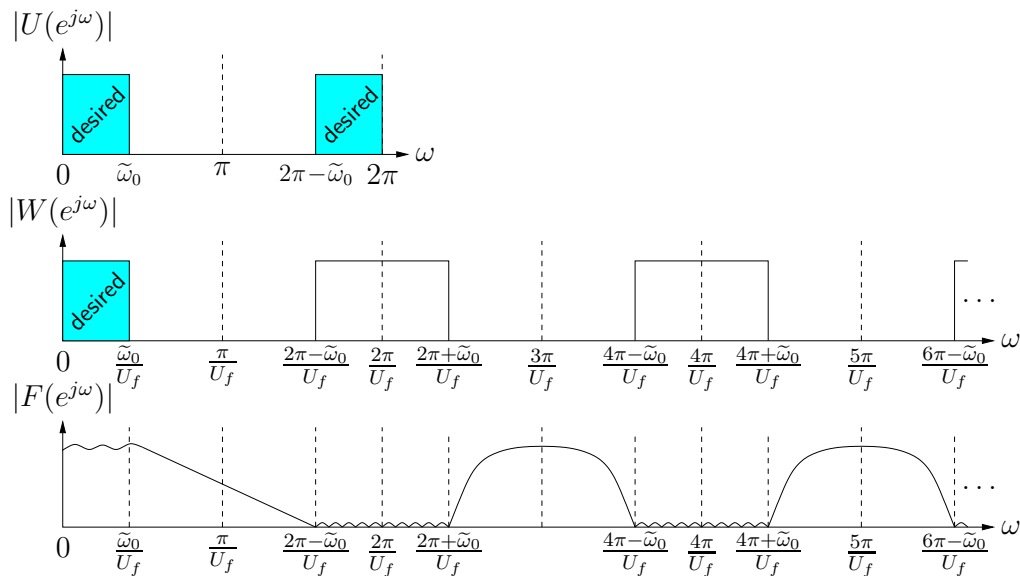
- Assume $x[n]$ is bandlimited to $\omega_0 < \pi$ rad/samp. Then...
 - $G(e^{j\omega})$ will have a passband cutoff at π/U_g and transition bandwidth $\Delta_g = \frac{2\pi - 2\omega_0}{U_g}$:



- Its output, $u[n]$, will be bandlimited to $\tilde{\omega}_0 \triangleq \frac{\omega_0}{U_g} < \frac{\pi}{2}$.

Multi-stage Interpolation (cont.):

- $F(e^{j\omega})$ will have a passband cutoff at π/U_f and transition bandwidth $\Delta_f = \frac{2\pi - 2\tilde{\omega}_0}{U_f} = \frac{2\pi U_g - 2\omega_0}{U}$.



- Say that δ_s and δ_p are defined relative to DC gain = 1. That is, we design unit-gain LPFs and then scale them.
- For the passband ripples, we want

$$U(1 + \delta_p) = A_g(1 + \delta_{p,g})A_f(1 + \delta_{p,f}) \approx U(1 + \delta_{p,g} + \delta_{p,f})$$
 and so we set $\delta_{p,g} = \delta_p/2$ and $\delta_{p,f} = \delta_p/2$.
- For the stopband ripples, we want

$$U\delta_s = \max \{ A_g \times A_f \delta_{s,f}, A_f \times A_g \delta_{s,g} \}$$
 and so we set $\delta_{s,g} = \delta_s$ and $\delta_{s,f} = \delta_s$.
- How do we choose U_g and A_g (and thus U_f and A_f)?

Computational Savings of Multi-stage Interp:

- Kaiser's formula can be used to approximate the required filter lengths. For the single-stage $H(z)$ we have

$$L_h \approx \frac{-10 \log_{10}(\delta_p \delta_s) - 13}{2.3 \Delta_h} \triangleq \frac{\beta}{2.3(2\pi - 2\omega_0)} U$$

using $\Delta_h = \frac{2\pi - 2\omega_0}{U}$ and $\beta \triangleq -10 \log_{10}(\delta_s \delta_p) - 13$.

Thus L_h mults/input with polyphase implementation.

- For 2-stage interpolation, we have

$$L_g \approx \frac{-10 \log_{10}(\delta_s \delta_p / 2) - 13}{2.3 \Delta_g} \approx \frac{\beta + 3}{2.3(2\pi - 2\omega_0)} U_g$$
$$L_f \approx \frac{-10 \log_{10}(\delta_s \delta_p / 2) - 13}{2.3 \Delta_f} \approx \frac{\beta + 3}{2.3(2\pi U_g - 2\omega_0)} U$$

And with polyphase implementations of both stages, the total number of mults-per-input equals $L_g + U_g L_f$.

- Thus, the ratio of 2-stage to 1-stage multiplies equals

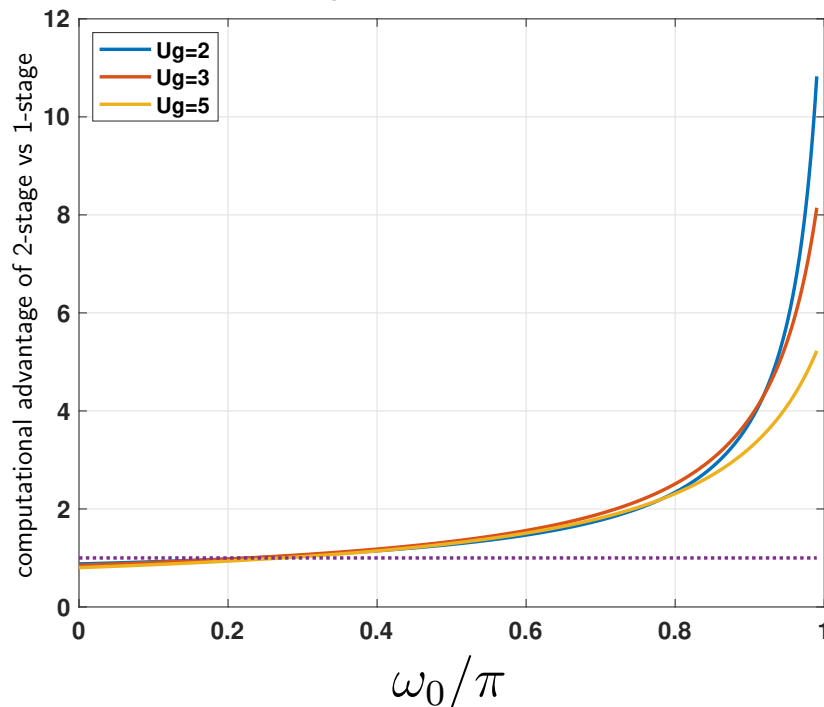
$$\frac{L_g + U_g L_f}{L_h} \approx \frac{\beta + 3}{\beta} \left(\frac{1}{U} + \frac{\pi - \omega_0}{\pi U_g - \omega_0} \right) U_g.$$

We can use this approximation to design U_g .

- Above eqns show that the choice of A_g doesn't matter.

Example of Multi-stage Interpolation:

- Suppose $U = 30$, $\delta_p = 0.002$, $\delta_s = 0.001$.
- The figure below shows the computational advantage of 2-stage over 1-stage. (Anything > 1 is an advantage.)



We see that...

- The advantage of using 2 stages grows as $\omega_0 \rightarrow \pi$.
- For large ω_0 , we should make U_g as small as possible.
- For medium or small ω_0 , choice of U_g doesn't matter.
- Above suggests little benefit to using 3 or more stages.
 - Little advantage to splitting 2nd stage because $\tilde{\omega}_0 < \frac{\pi}{2}$
- Similar ideas can be used for multi-stage decimation.